

DiscreteRTC: Discrete Diffusion Policies are Natural Asynchronous Executors

Pengcheng Wang¹, Kaiwen Hong², Chensheng Peng¹,
Katherine Driggs-Campbell², Masayoshi Tomizuka¹, Chenfeng Xu³, Chen Tang⁴
UC Berkeley¹, UIUC², UT Austin³, UCLA⁴
wangpc@berkeley.edu, cxu@utexas.edu, ctangac@ucla.edu

Abstract: Unlike chatbots, physical AI must act while the world keeps evolving. Therefore, the inter-chunk pause of synchronous executors is fatal for dynamic tasks regardless of how fast the inference is. Asynchronous execution—*thinking while acting*—is therefore a structural requirement, and real-time chunking (RTC) makes it viable by recasting chunk transitions as inpainting: freezing committed actions and consistently generating the remainder. However, RTC with a flow-matching policy is structurally suboptimal: its inpainting comes from inference-time corrections rather than the base policy, yielding little pre-training benefit, specific fine-tuning, heuristic guidance, and extra computation that inflates the latency. In this work, we observe that discrete diffusion policies, which generate actions by iteratively unmasking, are *natural asynchronous executors that resolve all limitations at once*: they are fine-tuning free since inpainting is their native operation, while early stopping further provides adaptive guidance and reduces inference cost. We propose **DiscreteRTC**, which replaces external corrections with native unmasking, and show on dynamic simulated benchmarks and real-world dynamic manipulation tasks that it achieves higher success rates than ContinuousRTC and other baselines. In summary, DiscreteRTC is simpler to implement with 0 lines of additional code to enable async inpainting, faster at inference with only $\sim 0.7\times$ computation compared with generating actions from scratch, and better at execution with 65% higher success rate in real-world hockey defend task compared with flow-matching RTC, and 30% higher compared with training-time flow-matching RTC. More visualizations are on our [website](#).

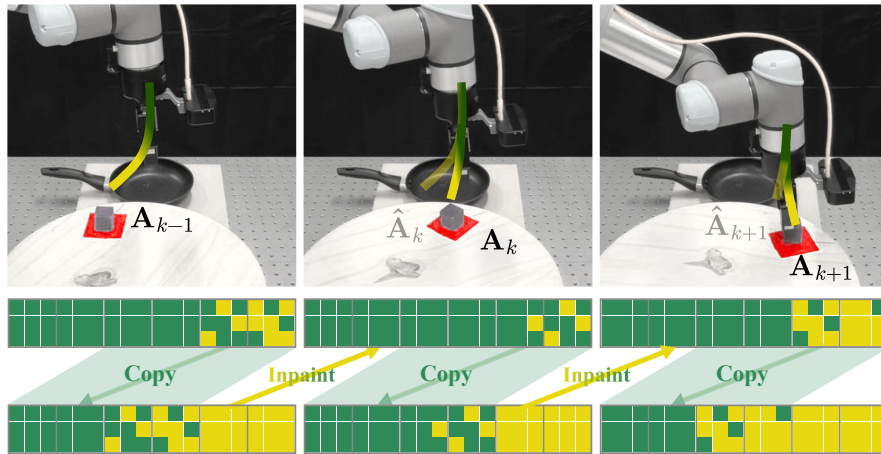


Figure 1: **Async Execution with discrete diffusion policies solving dynamic manipulation.** Gray rectangles and blocks represent the action chunks and the actions. Yellow and green cubes represent the masked and unmasked action tokens. During each inference cycle, discrete diffusion policies copy the tail of the last action chunk as the committed prefix, and inpaint upon it by simply forwarding itself. Compared with flow-matching-based inpainting that relies on IIGDM, discrete-diffusion-based inpainting inference is simpler to implement, faster at inference, and better at execution.

1 Introduction

Unlike chatbots [1, 2] or image generators [3], physical AI systems [4, 5, 6, 7, 8, 9, 10, 11, 12, 13] act with the world evolving: the robot cannot afford to pause while thinking. This reveals a structural limitation of synchronous execution with action-chunking policies [14, 15]—every inference call forces a mandatory pause between chunks during which the robot is idle but the world is not, which is fatal for dynamic tasks such as tracking moving targets [16]. Therefore, asynchronous execution—*thinking while acting* over action chunks—is not a latency optimization but a structural requirement.

However, naive async strategies, whether switching chunks immediately or smoothing them via temporal ensembling [17], introduce inter-chunk discontinuity or invalid actions because each new chunk is generated in isolation from the committed actions. Real-time chunking (RTC) [18] resolves this by recasting chunk transitions as an *inpainting* problem: freezing committed actions from the previous chunk and consistently generating the remainder, representing a simple yet effective approach that has achieved great success in precise and smooth real-world manipulation [19, 20].

Specifically, RTC is specially designed for flow-matching policies [21]. As the predominant action head architecture in today’s state-of-the-art VLAs [4, 5, 6], flow-matching has driven much of the recent progress in large-scale robot learning. However, in this paper, we first systematically show that *RTC with a flow-matching head is far from ideal with four critical limitations*:

- (a) **Pre-training w/o Inpainting.** Flow-matching policies are not pre-trained with inconsistent noise for inpainting. Therefore, scaling pre-training does not directly improve asynchronous performance, and inference-time corrections [22] are inevitable for inpainting;
- (b) **Fine-tuning Required.** As a consequence, adequate inpainting quality demands a dedicated fine-tuning stage with techniques such as action-suffix conditioning [23, 24] to explicitly introduce the inpainting-specific noise pattern into training;
- (c) **Heuristic Guidance.** Moreover, to better leverage the previous action chunks, IIGDM requires a heuristic schedule fixed across different inference-time cases;
- (d) **Extra Inference Cost.** Finally, the correction guidance term at every denoising step roughly doubles inference cost, ironically increasing the very latency RTC aims to hide.

Many ongoing efforts [25, 26, 23, 27] seek to resolve these aforementioned issues within the flow-matching paradigm. Instead, our key insight and observation are that by replacing the action head with a discrete diffusion policy [28], all the aforementioned limitations can be resolved at once. Or, to put it simply: *Discrete Diffusion Policies are Natural Asynchronous Executors*:

- (a) **Inpainting as Pre-training.** Discrete diffusion policies are pre-trained to inpaint upon randomly masked sequences. Therefore, scaling pre-training directly improves asynchronous performance, and the native forward pass suits inference-time inpainting;
- (b) **Fine-tuning Free.** As a consequence, inpainting-specific patterns are implicitly introduced during pre-training, making discrete diffusion a fine-tuning-free approach for high-quality, out-of-the-box asynchronous execution;
- (c) **Natural Guidance.** Moreover, with discrete diffusion policies, we can early-exit inference once the necessary action tokens are unmasked, leaving the remaining masking pattern as an adaptive and natural guidance for the next inference;
- (d) **Lower Inference Cost.** Finally, with committed tokens from previous chunks, the tokens to unmask per inference are reduced, leading to lower inference cost for inpainting.

We propose the resulting async execution method, **DiscreteRTC**, which replaces the external corrections in flow-matching-based RTC with the native unmasking operation of discrete diffusion policies. Experiments on the dynamic simulated benchmark Kinetix [29] and real-world dynamic manipulations demonstrate that DiscreteRTC achieves higher success rates than flow-matching-based RTC and training-time baselines. In summary, DiscreteRTC is simpler to implement with 0 lines of code to enable async inpainting, faster at inference with only $\sim 0.7\times$ computation compared with generating actions from scratch while flow-matching-based RTC requires $\sim 1.7\times$ computation, and better at execution with 65% higher success rate in real-world hockey defend task compared with flow-matching RTC, and 30% higher success rate compared with training-time flow-matching RTC.

2 Preliminaries

Action Chunking Flow Policy. We consider an action chunking policy π that receives an observation $\mathbf{o}_t$ and outputs a chunk of H future actions $\mathbf{A}_t = (a_t, a_{t+1}, \dots, a_{t+H-1})$. State-of-the-art VLAs [4, 5] generate action chunks using conditional flow matching [21, 30]: noise $\mathbf{A}_t^0 \sim \mathcal{N}(0, I)$ is sampled and integrated through the learned velocity field v_π from $\tau = 0$ to 1:

$$\mathbf{A}_t^{\tau+\frac{1}{n}} = \mathbf{A}_t^\tau + \frac{1}{n} v_\pi(\mathbf{A}_t^\tau, \mathbf{o}_t, \tau), \quad (1)$$

where $\tau \in [0, 1)$ is the flow timestep and n is the number of denoising steps. During training, all actions within a chunk are corrupted at the same noise level τ and denoised together.

Real-Time Chunking. Running modern VLAs with action generation time δ smaller than the controller period Δt (typically 20~50 ms) is challenging [18]: even a 7B OpenVLA model optimized for speed [31, 32] requires $\delta = 321$ ms on a server-grade A100 GPU. The robot must therefore act while the policy is still computing, *i.e.*, asynchronous execution. Naively switching to a new action chunk as soon as it is ready, however, leads to inter-chunk discontinuity: adjacent chunks may correspond to different behavioral modes, producing jerky motions at chunk boundaries.

Real-time chunking (RTC) [18] addresses this discontinuity by posing asynchronous execution as an *inpainting* problem. If inference takes d (inference delay) steps to complete, the first d actions of the new chunk are “frozen” to the values from the previous chunk before the new chunk becomes available. The remaining actions must be generated to be consistent with this frozen prefix. For flow-matching policies, RTC implements this inpainting via IIGDM [22], adding a gradient-based correction at each denoising step that steers the generation toward the target prefix $\mathbf{Y}$:

$$\mathbf{g} = \left(\mathbf{Y} - \hat{f}_{\mathbf{A}^1}(\mathbf{A}^\tau) \right)^\top \text{diag}(\mathbf{W}) \cdot \left. \frac{\partial \hat{f}_{\mathbf{A}^1}}{\partial \mathbf{A}'} \right|_{\mathbf{A}^\tau}, \quad (2)$$

where $\hat{f}_{\mathbf{A}^1}$ denotes the one-step denoising function and $\mathbf{W}$ is a vector of soft-masking weights. The weights $\mathbf{W}$ assign 1 to the first d frozen actions, exponentially decay for the overlapping region, and 0 for the final s (execution horizon) actions. During inference, the robot starts an inference every s steps and executes the next s actions from the most recent inference results.

Discrete Diffusion Policies. Discrete diffusion policies [28] tokenize continuous actions into discrete tokens via a vocabulary (*e.g.*, VQ-VAE [33], FAST [34] or k -bin quantization). With a slight abuse of notation, we reuse $\mathbf{A}_t$ to denote the resulting action token sequence. The forward process replaces tokens with a special [MASK] token with a masking schedule, while the reverse process iteratively executes unmasking operations conditioned on the current partially masked sequence.

A discrete diffusion policy is trained for one-step unmasking. It consists of two components: p_π , which predicts the probability distribution over all token positions, and f_π , which samples which tokens to unmask at the current step from that distribution. At each step k , the policy first predicts token identities via p_π and then selects which tokens to unmask via f_π given the observation $\mathbf{o}_t$:

$$\mathbf{A}_t^{k+1} = \pi(\mathbf{A}_t^k, \mathbf{o}_t) = f_\pi(p_\pi(\mathbf{A}_t^k, \mathbf{o}_t)), \quad (3)$$

where $\mathbf{A}_t^k$ is the partially masked sequence at step k . The discrete diffusion policy learns to predict the fully unmasked ground-truth action token sequence given the randomly masked token sequence.

3 Flow-Matching is not Suitable for RTC

3.1 Pre-training without Inpainting

As illustrated in Figure 2, pre-training of flow-matching policies [21] learns to generate conditioned on observations and a noised action chunk. During training, all actions within a chunk are corrupted at the **consistent** noise level τ and are denoised together. However, during RTC, the inpainting starts from an action chunk with **inconsistent** noise levels across positions: 0 for the committed actions, 1

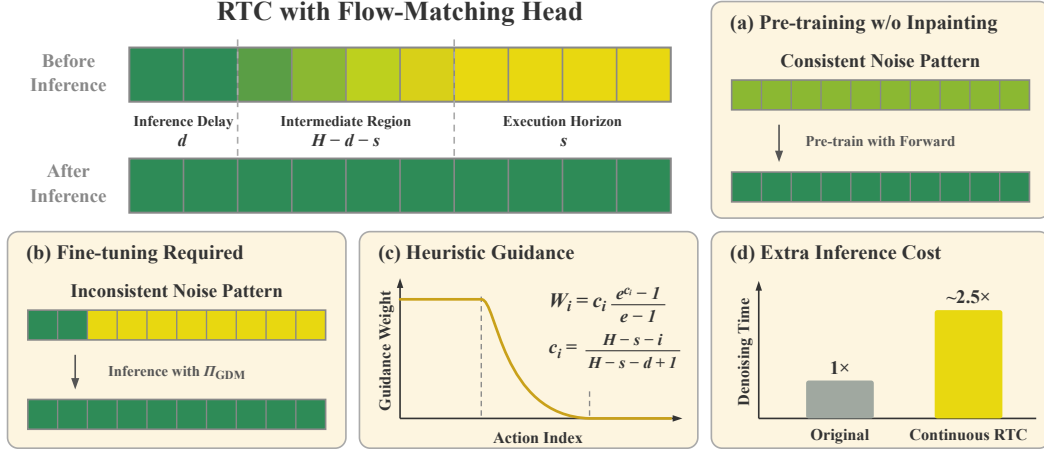


Figure 2: **RTC with flow-matching head.** Color represents the noise level, where green stands for the clear action and yellow stands for the pure noise. The flow-matching head is ill-suited for RTC because (a) during pre-training, the base policy is not trained on inpainting tasks; (b) to acquire this capability, a specially designed fine-tuning stage is required; (c) at inference time, RTC relies on heuristic guidance weight design; and (d) incurs extra cost for online correction computation.

for the new actions, and interpolated values for the intermediate actions. Although the Π_{GDM} [22] guidance provides an external correction at inference time, the model has encountered neither such corrections nor the mixed-noise chunk pattern during the pre-training stage. As a result, scaling up pre-training data or model capacity does not directly improve inpainting quality.

3.2 Fine-tuning Required

Following Sec. 3.1, to improve the inpainting capability of the base policy, the mixed-noise chunk pattern must be explicitly introduced during training along with a corresponding inpainting objective. This requires fine-tuning the model with an explicit inpainting loss [23], adding training complexity and risking interference with the base generation quality [24]. This introduces a non-trivial additional engineering and computational burden, particularly for large VLAs that are expensive [35] or even risky to fine-tune without degrading the base generation abilities [24].

3.3 Heuristic Guidance

To effectively inpaint with flow-matching policies, online corrections like Π_{GDM} [22] are inevitable. Completing Equation 2, the full denoising step of Π_{GDM} is presented as follows:

$$\mathbf{A}_t^{\tau + \frac{1}{n}} = \mathbf{A}_t^\tau + \frac{1}{n} \left(v_\pi(\mathbf{A}_t^\tau, \mathbf{o}_t, \tau) + \min\left(\beta, \frac{1 - \tau}{\tau \cdot r_\tau^2}\right) \left(\mathbf{Y} - \hat{f}_{\mathbf{A}^1}(\mathbf{A}^\tau) \right)^\top \text{diag}(\mathbf{W}) \frac{\partial \hat{f}_{\mathbf{A}^1}}{\partial \mathbf{A}'} \bigg|_{\mathbf{A}^\tau} \right). \quad (4)$$

Notably, the soft-masking weights W in Equation 4 follow a hand-crafted exponential decay schedule. This important guidance schedule, which decides how to follow the previous action chunks, is fixed across different inference situations and is validated only through empirical studies. This heuristic design not only creates a tuning burden, but also behaves inflexibly in response to different inference cases during asynchronous execution.

3.4 Extra Inference Cost

Finally, running Π_{GDM} at inference time requires non-negligible extra computation. At each denoising step, computing the guidance term in Equation 4 requires a vector-Jacobian product (VJP), which roughly doubles the per-step computation [24]. As a mechanism intended to enable smooth real-time execution, flow-matching-based RTC instead introduces additional inference delay, shrinking the time window available for generating new actions and counteracting its design goal.

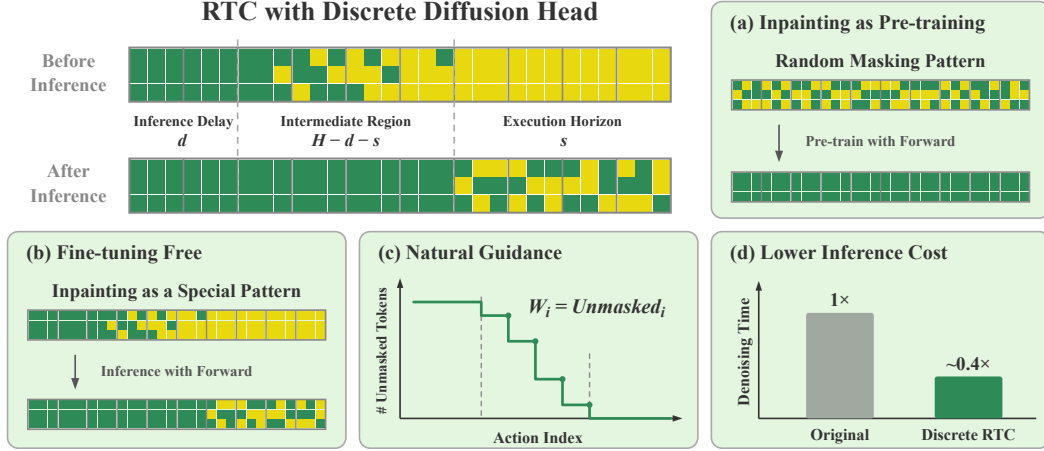


Figure 3: **RTC with discrete diffusion head**. Color represents the masking status, where green stands for the unmasked token and yellow stands for the masked token. The discrete diffusion head is naturally suited for RTC because (a) during pre-training, the base policy is already trained on inpainting tasks; (b) consequently, no inpainting-specific fine-tuning is required; (c) at inference time, early stopping from the previous inference leaves a natural guidance signal; and (d) fewer tokens need to be unmasked per step, reducing the computational cost of each forward pass.

4 Discrete Diffusion Policies are Natural Asynchronous Executors

As illustrated in Figure 3, we show that discrete diffusion policies can naturally resolve all the aforementioned limitations of flow-matching-based RTC at once.

4.1 Learning to Inpaint from Pre-training without Fine-tuning

In discrete diffusion, inpainting is the native operation: given a partially masked token sequence, the policy is asked to reconstruct the target action chunks by predicting the masked tokens. Therefore, pre-training directly improves the model’s ability to inpaint at inference time: scaling model, data, and pre-training computation yields better asynchronous execution without inpainting-specific fine-tuning. To inpaint at inference time, no extra implementation, loss term, or inference-time guidance is needed:

$$\mathbf{A}_t^{k+1} = \pi(\mathbf{A}_t^k, \mathbf{o}_t). \quad (5)$$

Unlike Equation 4, the model generates conditioned on the partially unmasked token sequence from the last inference, exactly as it was trained, by simply forwarding the base policy.

4.2 Natural Guidance

To solve the guidance issue, we exploit a key property of discrete diffusion: once a token is unmasked, it already carries a well-defined semantic meaning. This stands in contrast to continuous diffusion or flow-matching, where the denoised action chunk becomes valid and executable all together only when the noise level reaches zero at the end of the denoising process. Therefore, with discrete diffusion, the model can not only start inference from an intermediate state (e.g. a partially unmasked action chunk with prefix), but also terminate at an intermediate stage.

To ensure valid execution before the next inference, only the next s actions after the committed actions are necessary to be fully unmasked. Therefore, the inference can early-exit when satisfying the minimal criteria, leaving the remaining actions a partially unmasked pattern. By carrying over the results, this pattern serves as a *natural and adaptive* guidance for the next inference with the number of the unmasked tokens, replacing the heuristic and fixed guidance in flow-matching RTC.

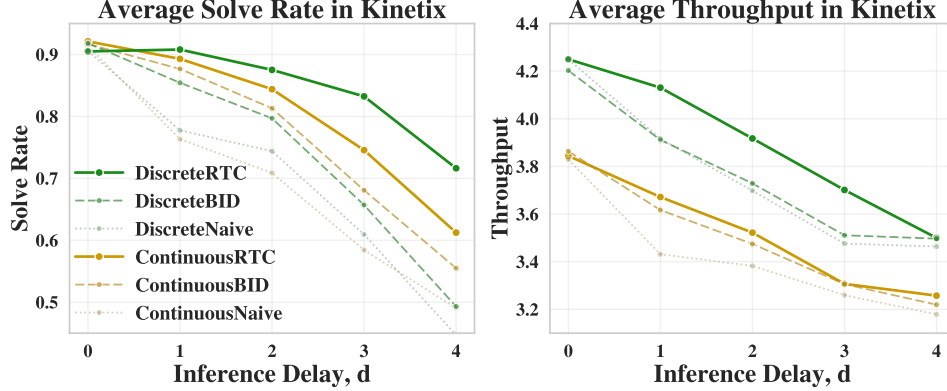


Figure 4: **Experimental Results in Kinetix.** The throughputs represent the task completed by the policy every 256 steps. **Left:** Average solve rate and throughputs across all environments with different inference delays; **Right:** Solve rates for every tasks with different inference delays. The execution horizon follows $s = \max(1, d)$ and each datapoint represents 2048 trials.

4.3 Lower Inference Cost

Finally, since unmasking starts from an intermediate state and ends at an intermediate stage, keeping the number of tokens unmasked per step constant actually reduces the total number of tokens to unmask per inference to roughly s/H of the original, or at least $1 - d/H$ by starting only from the actions to execute and terminating at complete action chunks. This means that, instead of *increasing* inference cost as in flow-matching-based RTC, our method can actually *reduce* it by simply forwarding the base policy with fewer iterative steps. Alternatively, we can keep the number of inference steps fixed and reduce the number of tokens unmasked per step, thereby producing finer-grained actions without increasing the action generation cost compared with generation from scratch.

5 Experiments

5.1 Simulated Benchmark

We first evaluate DiscreteRTC in Kinetix [29], a simulated benchmark of dynamic tasks. To simulate imperfect actuation, we add Gaussian noise to the actions to make closed-loop corrections crucial for success [24]. Please refer to Appendix B for additional experimental results.

Setup. We implemented the discrete diffusion policies upon the official codebase and datasets from RTC [18]. For fair comparison, we adopt the same training setup (optimizer, learning rate, epochs, etc.) and the same policy network size except the final layer for logits output. We use a trivial 512-bin quantization for the action chunk discretization. Please refer to Appendix C for more details.

Baselines. With the same policy architecture, we compare against the flow-matching (Continuous) and the discrete diffusion (Discrete) action head under the following asynchronous strategies:

- **Naive Async.** Chunks are generated from scratch and switched when the new one is ready.
- **Bidirectional decoding (BID)** [36]. Uses rejection sampling to maintain cross-chunk continuity. We use a batch size of $N = 16$ with no weak policy as the official implementation.
- **RTC** [18]. Flow-matching policy with IIGDM inpainting and soft masking, representing the state-of-the-art asynchronous execution method for continuous policies.

Results. We present the main results in Kinetix in Figure 4. Under different inference delays, DiscreteRTC consistently outperforms ContinuousRTC and other variants on both solve rates and throughputs, showing the advantage of the native inpainting capability in discrete diffusion policies.

We present the extended experimental results in Kinetix in Figure 5. As shown in the results of iterative steps per policy, DiscreteRTC requires fewer iterative steps than ContinuousRTC, espe-

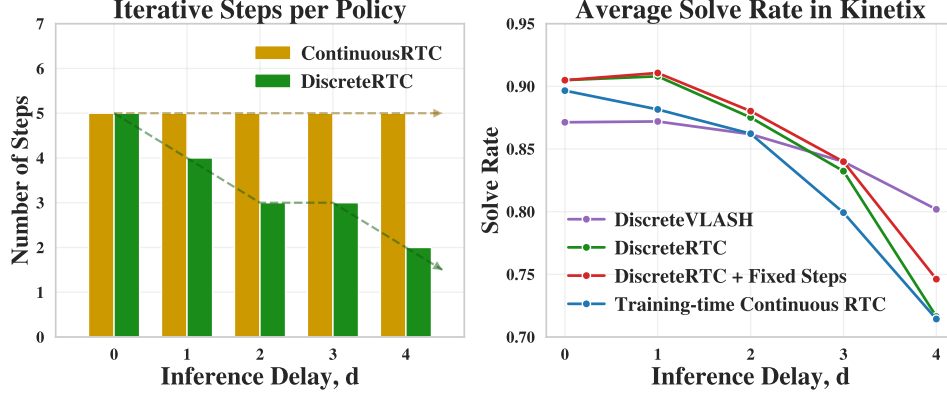


Figure 5: **Extended Experimental Results in Kinetix.** **Left:** Required iterative steps for each inpainting inference of different policy architectures in Kinetix with $s = \max(1, d)$; **Right:** Average solve rates of extended variants in Kinetix. The evaluation setup keeps the same with Figure 4.

cially under large delays with execution horizon as discussed in Sec. 4. Moreover, regarding the average solve rates of extended variants in Kinetix, we show three important observations: (1) First, DiscreteRTC can outperform *Training-time ContinuousRTC* [23], which is fine-tuned with an explicit objective and recipe to improve inpainting, further demonstrating the strong performance of the fine-tuning free DiscreteRTC. (2) Next, by using fixed unmasking steps during inference, *DiscreteRTC + Fixed Steps* can achieve better performance by generating more fine-grained actions with the same computation budget as discussed in Sec. 4.3. (3) Moreover, *DiscreteVLASH*, which integrates the VLASH [37] can achieve more stable performance across varying delays but at the cost of performance drop under small delays. This demonstrates that discrete diffusion can be seamlessly combined with advanced inference-time methods for further gains.

5.2 Real-World Results

Next, we validate DiscreteRTC on a real robot platform, focusing on dynamic tasks where inference latency and control frequency have a crucial impact on the closed-loop performance. Please refer to Appendix D for full deployment details and visualizations.

Setup. We use a UR5e arm with a Robotiq gripper and a wrist-mounted RGB camera. Developed with StarVLA [38], all policies share a Qwen2.5-VL-3B-Instruct [2] VLM paired with a layerwise cross-attention DiT [39] action head. The robot operates on a single RTX 4090. Regarding the baselines, we mainly consider the Sync, RTC, and training-time RTC with flow-matching and discrete diffusion heads to better demonstrate our main claims. We design three dynamic tasks to directly stress the reactivity, **Dynamic Pick**, **Dynamic Place**, and **Hockey Defend**. Regarding the former two tasks, the robot is asked to pick a moving object or place the object on a moving platform; for the last, the robot is asked to defend a high-speed hockey attack from the opponent policy. These tasks require the robot to keep re-estimating the object pose and to execute agilely and smoothly.

Results. Figure 6 summarizes the real-world results. Both Sync baselines completely fail (0%), confirming that reactive asynchronous execution is indispensable in these regimes. In terms of success rate, DiscreteRTC outperforms ContinuousRTC with a huge gap on the Hockey Defend and the Dynamic Pick where the reactivity is critical for success. In terms of inference time, the two asynchronous methods move in opposite directions: ContinuousRTC inflates flow-matching cost with a near- $1.7\times$ overhead from the IIGDM, whereas DiscreteRTC *reduces* discrete diffusion cost to around $0.7\times$ since fewer tokens need to be unmasked during inference. Consequently, DiscreteRTC ends up both faster and better than ContinuousRTC. Even compared with training-time ContinuousRTC with extra fine-tuning efforts, DiscreteRTC can still outperform with higher action qualities and success rates. However, the naive k-bin tokenization blocks DiscreteRTC from being more efficient than the training-time methods, a limitation discussed in the final section.

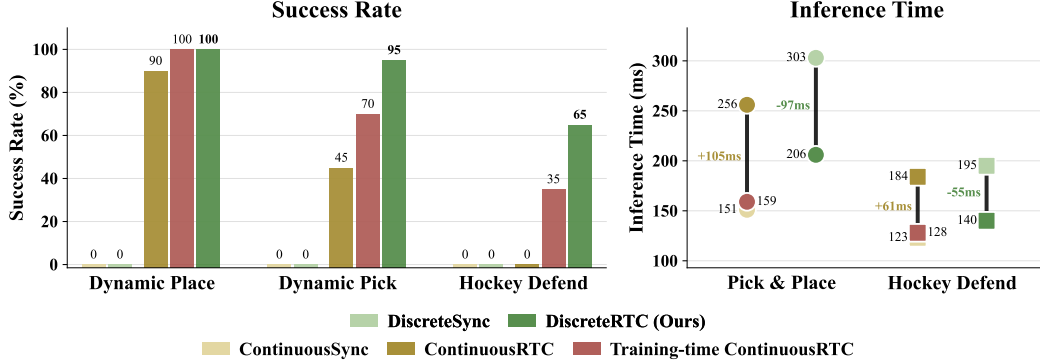


Figure 6: Real-world results on the three dynamic tasks. We report task success rate over 20 trials per task and average inference time (ms) on a single RTX 4090.

6 Related Work

We briefly review the most relevant works in this section with an extended discussion in Appendix A.

Efficient VLA via Discrete Diffusion. Prior efforts to accelerate VLAs span system-level optimization [40] and action tokenization [41, 42, 5]. However, tokenization-based methods still rely on autoregressive decoding at inference time, limiting throughput. Discrete diffusion [43] replaces autoregressive generation with parallel unmasking, and several recent works apply it to VLA action heads [28, 44, 45, 46, 47, 48, 49]. While these works demonstrate the efficiency of discrete diffusion, none investigate its structural advantage for asynchronous execution, which is the focus of ours.

Asynchronous Execution Strategy. When inference latency exceeds the control period, the robot must act while computing [50]. With action chunking, strategies range from naive chunk switching and temporal ensembling [17] to rejection-based methods [36]. Real-time chunking (RTC) [18] poses chunk transitions as inpainting via IIGDM guidance [22], with follow-up works improving guidance design [27, 26] and learning inpainting at training time [23, 24, 25]. However, all of these methods operate within the flow-matching framework, while DiscreteRTC is the first to exploit the native inpainting capability of discrete diffusion policies for natural asynchronous execution.

7 Discussion and Future Steps

Conclusions. We presented DiscreteRTC, which exploits the native inpainting capability of discrete diffusion for asynchronous real-time control, eliminating the need for inpainting-specific fine-tuning, heuristic guidance weights, and extra inference cost inherent to flow-matching-based RTC. Simulated and real-world experiments show that DiscreteRTC outperforms ContinuousRTC and training-time baselines, and can be combined with methods such as VLASH for further gains.

Limitations. We note several limitations of our current instantiation, each of which coincides with an active research direction that benefits DiscreteRTC. (1) Our naive k -bin quantization yields overly long token sequences, motivating compact, temporally aware tokenizers [41, 42]. (2) Our modularized AR-VLM + discrete diffusion head prevents the backbone from participating in iterative unmasking, a gap that unified discrete-diffusion VLAs [44, 45, 46, 47] aim to close. (3) Traditional max-confidence unmasking does not yet translate the natural schedule into consistent gains.

Future Steps. These limitations point to three concrete future research directions: 1) A *time-causally ordered action tokenizer* that produces temporally ordered, compact token representations; 2) A *unified discrete diffusion VLA* in which observation reasoning and action generation share the same backbone; and 3) An *appropriate yet principled unmasking strategy* that fully unleashes the potential of the natural schedule. Emerging techniques such as AR-block decoding align better with the implicit autoregressive structure induced by RTC and integrate seamlessly.

References

- [1] T. Brown, B. Mann, N. Ryder, M. Subbiah, J. D. Kaplan, P. Dhariwal, A. Neelakantan, P. Shyam, G. Sastry, A. Askell, et al. Language models are few-shot learners. *Advances in neural information processing systems*, 33:1877–1901, 2020.
- [2] A. Yang, A. Li, B. Yang, B. Zhang, B. Hui, B. Zheng, B. Yu, C. Gao, C. Huang, C. Lv, et al. Qwen3 technical report. *arXiv preprint arXiv:2505.09388*, 2025.
- [3] T. Wan, A. Wang, B. Ai, B. Wen, C. Mao, C.-W. Xie, D. Chen, F. Yu, H. Zhao, J. Yang, et al. Wan: Open and advanced large-scale video generative models. *arXiv preprint arXiv:2503.20314*, 2025.
- [4] K. Black, N. Brown, D. Driess, A. Esmail, M. Equi, C. Finn, N. Fusai, L. Groom, K. Hausman, B. Ichter, et al. pi0: A vision-language-action flow model for general robot control. *arXiv preprint arXiv:2410.24164*, 2024.
- [5] P. Intelligence, K. Black, N. Brown, J. Darpinian, K. Dhabalia, D. Driess, A. Esmail, M. Equi, C. Finn, N. Fusai, et al. pi0.5: a vision-language-action model with open-world generalization. *arXiv preprint arXiv:2504.16054*, 2025.
- [6] J. Bjorck, F. Castañeda, N. Cherniadev, X. Da, R. Ding, L. Fan, Y. Fang, D. Fox, F. Hu, S. Huang, et al. Gr00t n1: An open foundation model for generalist humanoid robots. *arXiv preprint arXiv:2503.14734*, 2025.
- [7] P. Wang, Q. Liu, H. Lin, Y. Li, G. Zhan, M. Tomizuka, and Y. Wang. Dadp: Domain adaptive diffusion policy. *arXiv preprint arXiv:2602.04037*, 2026.
- [8] OpenAI, M. Andrychowicz, B. Baker, M. Chociej, R. Józefowicz, B. McGrew, J. Pachocki, A. Petron, M. Plappert, G. Powell, A. Ray, J. Schneider, S. Sidor, J. Tobin, P. Welinder, L. Weng, and W. Zaremba. Learning dexterous in-hand manipulation. *CoRR*, 2018. URL <http://arxiv.org/abs/1808.00177>.
- [9] S. An, Z. Meng, C. Tang, Y. Zhou, T. Liu, F. Ding, S. Zhang, Y. Mu, R. Song, W. Zhang, et al. Dexterous manipulation through imitation learning: A survey. *arXiv preprint arXiv:2504.03515*, 2025.
- [10] Z. Su, B. Zhang, N. Rahmanian, Y. Gao, Q. Liao, C. Regan, K. Sreenath, and S. S. Sastry. Hitter: A humanoid table tennis robot via hierarchical planning and learning. *arXiv preprint arXiv:2508.21043*, 2025.
- [11] R. Yu, Y. Wang, Q. Zhao, H. W. Tsui, J. Wang, P. Tan, and Q. Chen. Skillmimic-v2: Learning robust and generalizable interaction skills from sparse and noisy demonstrations. In *Proceedings of the Special Interest Group on Computer Graphics and Interactive Techniques Conference Conference Papers*, pages 1–11, 2025.
- [12] J. Hu, J. Shim, C. Tang, Y. Sung, B. Liu, P. Stone, and R. Martin-Martin. Simple recipe works: Vision-language-action models are natural continual learners with reinforcement learning. *arXiv preprint arXiv:2603.11653*, 2026.
- [13] Z. Huang, Y. Zhang, J. Liu, R. Song, C. Tang, and J. Ma. Tic-vla: A think-in-control vision-language-action model for robot navigation in dynamic environments. *arXiv preprint arXiv:2602.02459*, 2026.
- [14] C. Chi, Z. Xu, S. Feng, E. Cousineau, Y. Du, B. Burchfiel, R. Tedrake, and S. Song. Diffusion policy: Visuomotor policy learning via action diffusion. *The International Journal of Robotics Research*, 44(10-11):1684–1704, 2025.
- [15] L. Lai, A. Z. Huang, and S. J. Gershman. Action chunking as conditional policy compression. *Cognition*, 264:106201, 2025.

- [16] H. Xie, B. Wen, J. Zheng, Z. Chen, F. Hong, H. Diao, and Z. Liu. Dynamicvla: A vision-language-action model for dynamic object manipulation. *arXiv preprint arXiv:2601.22153*, 2026.
- [17] T. Z. Zhao, V. Kumar, S. Levine, and C. Finn. Learning fine-grained bimanual manipulation with low-cost hardware. *arXiv preprint arXiv:2304.13705*, 2023.
- [18] K. Black, M. Y. Galliker, and S. Levine. Real-time execution of action chunking flow policies. *arXiv preprint arXiv:2506.07339*, 2025.
- [19] P. Intelligence, A. Amin, R. Aniceto, A. Balakrishna, K. Black, K. Conley, G. Connors, J. Darpinian, K. Dhabalia, J. DiCarlo, et al. $\pi_{0.6}^*$: a vla that learns from experience. *arXiv preprint arXiv:2511.14759*, 2025.
- [20] P. Intelligence, B. Ai, A. Amin, R. Aniceto, A. Balakrishna, G. Balke, K. Black, G. Bokinsky, S. Cao, T. Charbonnier, et al. $\pi_{0.7}$: a steerable generalist robotic foundation model with emergent capabilities. *arXiv preprint arXiv:2604.15483*, 2026.
- [21] Y. Lipman, R. T. Chen, H. Ben-Hamu, M. Nickel, and M. Le. Flow matching for generative modeling. *arXiv preprint arXiv:2210.02747*, 2022.
- [22] J. Song, A. Vahdat, M. Mardani, and J. Kautz. Pseudoinverse-guided diffusion models for inverse problems. In *International conference on learning representations*, 2023.
- [23] K. Black, A. Z. Ren, M. Equi, and S. Levine. Training-time action conditioning for efficient real-time chunking. *arXiv preprint arXiv:2512.05964*, 2025.
- [24] H. Wang, G. Zhang, Y. Yan, Y. Shang, R. R. Kompella, and G. Liu. Real-time robot execution with masked action chunking. *arXiv preprint arXiv:2601.20130*, 2026.
- [25] Y. Liu, H. Yu, J. Zhao, B. Li, D. Zhang, M. Li, W. Wu, Y. Hu, J. Xie, J. Guo, et al. Learning native continuation for action chunking flow policies. *arXiv preprint arXiv:2602.12978*, 2026.
- [26] Y. Lu, Z. Liu, X. Fan, Z. Yang, J. Hou, J. Li, K. Ding, and H. Zhao. Faster: Rethinking real-time flow vlas. *arXiv preprint arXiv:2603.19199*, 2026.
- [27] F. Yang, P. Jing, K. Qu, N. Zhao, and Y. Su. Abpolicy: Asynchronous b-spline flow policy for real-time and smooth robotic manipulation. *arXiv preprint arXiv:2602.23901*, 2026.
- [28] Z. Liang, Y. Li, T. Yang, C. Wu, S. Mao, T. Nian, L. Pei, S. Zhou, X. Yang, J. Pang, et al. Discrete diffusion vla: Bringing discrete diffusion to action decoding in vision-language-action policies. *arXiv preprint arXiv:2508.20072*, 2025.
- [29] M. Matthews, M. Beukman, C. Lu, and J. Foerster. Kinetix: Investigating the training of general agents through open-ended physics-based control tasks. *arXiv preprint arXiv:2410.23208*, 2024.
- [30] G. Zhan, L. Tao, P. Wang, Y. Wang, Y. Li, Y. Chen, H. Li, M. Tomizuka, and S. E. Li. Mean flow policy with instantaneous velocity constraint for one-step action generation. *arXiv preprint arXiv:2602.13810*, 2026.
- [31] M. J. Kim, C. Finn, and P. Liang. Fine-tuning vision-language-action models: Optimizing speed and success. *arXiv preprint arXiv:2502.19645*, 2025.
- [32] M. J. Kim, K. Pertsch, S. Karamcheti, T. Xiao, A. Balakrishna, S. Nair, R. Rafailov, E. Foster, G. Lam, P. Sanketi, et al. Openvla: An open-source vision-language-action model. *arXiv preprint arXiv:2406.09246*, 2024.
- [33] A. Van Den Oord, O. Vinyals, et al. Neural discrete representation learning. *Advances in neural information processing systems*, 30, 2017.

- [34] K. Pertsch, K. Stachowicz, B. Ichter, D. Driess, S. Nair, Q. Vuong, O. Mees, C. Finn, and S. Levine. Fast: Efficient action tokenization for vision-language-action models. *arXiv preprint arXiv:2501.09747*, 2025.
- [35] E. J. Hu, Y. Shen, P. Wallis, Z. Allen-Zhu, Y. Li, S. Wang, L. Wang, W. Chen, et al. Lora: Low-rank adaptation of large language models. *Iclr*, 1(2):3, 2022.
- [36] Y. Liu, J. I. Hamid, A. Xie, Y. Lee, M. Du, and C. Finn. Bidirectional decoding: Improving action chunking via closed-loop resampling. *arXiv preprint arXiv:2408.17355*, 2024.
- [37] J. Tang, Y. Sun, Y. Zhao, S. Yang, Y. Lin, Z. Zhang, J. Hou, Y. Lu, Z. Liu, and S. Han. Vlash: Real-time vlas via future-state-aware asynchronous inference. *arXiv preprint arXiv:2512.01031*, 2025.
- [38] J. Ye, N. Gao, S. Yang, J. Zheng, Z. Wang, Y. Chen, P. Chen, Y. Chen, S. Liu, and J. Jia. Starvla- α : Reducing complexity in vision-language-action systems. *arXiv preprint arXiv:2604.11757*, 2026.
- [39] W. Peebles and S. Xie. Scalable diffusion models with transformers. In *Proceedings of the IEEE/CVF international conference on computer vision*, pages 4195–4205, 2023.
- [40] Y. Ma, Y. Zhou, Y. Yang, T. Wang, and H. Fan. Running vlas at real-time speed. *arXiv preprint arXiv:2510.26742*, 2025.
- [41] K. Pertsch, K. Stachowicz, B. Ichter, D. Driess, S. Nair, Q. Vuong, O. Mees, C. Finn, and S. Levine. Fast: Efficient action tokenization for vision-language-action models. *arXiv preprint arXiv:2501.09747*, 2025.
- [42] C. Liu, X. Han, J. Gao, Y. Zhao, H. Chen, and Y. Du. Oat: Ordered action tokenization. *arXiv preprint arXiv:2602.04215*, 2026.
- [43] A. Lou, C. Meng, and S. Ermon. Discrete diffusion modeling by estimating the ratios of the data distribution. *arXiv preprint arXiv:2310.16834*, 2023.
- [44] Y. Wen, H. Li, K. Gu, Y. Zhao, T. Wang, and X. Sun. Llada-vla: Vision language diffusion action models. *arXiv preprint arXiv:2509.06932*, 2025.
- [45] J. Wen, M. Zhu, J. Liu, Z. Liu, Y. Yang, L. Zhang, S. Zhang, Y. Zhu, and Y. Xu. dvla: Diffusion vision-language-action model with multimodal chain-of-thought. *arXiv preprint arXiv:2509.25681*, 2025.
- [46] J. Chen, W. Song, P. Ding, Z. Zhou, H. Zhao, F. Tang, D. Wang, and H. Li. Unified diffusion vla: Vision-language-action model via joint discrete denoising diffusion process. *arXiv preprint arXiv:2511.01718*, 2025.
- [47] J. Ye, S. Gong, J. Gao, J. Fan, S. Wu, W. Bi, H. Bai, L. Shang, and L. Kong. Dream-vl & dream-vla: Open vision-language and vision-language-action models with diffusion language model backbone. *arXiv preprint arXiv:2512.22615*, 2025.
- [48] J. Chen, W. Song, S. Chen, J. Wang, Z. Li, and H. Li. Dfm-vla: Iterative action refinement for robot manipulation via discrete flow matching. *arXiv preprint arXiv:2603.26320*, 2026.
- [49] W. Song, J. Chen, S. Chen, J. Wang, P. Ding, H. Zhao, Y. Qin, X. Zheng, D. Wang, Y. Wang, et al. Fast-dvla: Accelerating discrete diffusion vla to real-time performance. *arXiv preprint arXiv:2603.25661*, 2026.
- [50] T. Xiao, E. Jang, D. Kalashnikov, S. Levine, J. Ibarz, K. Hausman, and A. Herzog. Thinking while moving: Deep reinforcement learning with concurrent control. *arXiv preprint arXiv:2004.06089*, 2020.

- [51] J. Bradbury, R. Frostig, P. Hawkins, M. J. Johnson, C. Leary, D. Maclaurin, G. Necula, A. Paszke, J. VanderPlas, S. Wanderman-Milne, and Q. Zhang. JAX: composable transformations of Python+NumPy programs, 2018. URL <http://github.com/jax-ml/jax>.

A Extended Related Works

Efficient VLA via Discrete Diffusion. To train and run VLAs efficiently, many prior efforts have focused on system-level optimization [32], such as applying CUDA graph compilation and operator fusion to achieve real-time VLA inference on consumer GPUs [40]. Action tokenization is another promising direction that leverages the fully optimized VLM pipeline by converting continuous actions into discrete tokens [5]. Beyond trivial bin-based discretization, works like FAST [41] and OAT [42] compress action chunks into compact discrete token sequences, significantly reducing the required generation length. However, these tokenization-based approaches still rely on autoregressive decoding at inference time, which inherently limits parallelism and throughput [5]; as a result, many of them fall back to a continuous flow-matching action head for deployment and are only used for efficient large-scale VLA pre-training.

Discrete diffusion [43] offers a promising alternative by replacing autoregressive generation with parallel unmasking. Several recent works have begun exploring this direction: Liang et al. [28] apply discrete diffusion to the VLA action head, generating action chunks via iterative re-masking and parallel decoding; LLaDA-VLA [44] and DVLA [45] build unified discrete-diffusion VLA frameworks; UD-VLA [46] unifies observation reasoning and action generation within a single discrete diffusion backbone. Among them, DreamVLA [47] further replaces the VLM module with a discrete diffusion architecture, making it possible to pre-train the VLA with a unified backbone.

Asynchronous Execution Strategy. When inference latency exceeds the control period, the robot must act while the policy is still computing. The earliest formulation of this concurrent-control setting is Thinking While Moving [50], which executes the previous action during the current inference. With action chunking [15], the simplest strategy is naive asynchronous execution, which switches to a new chunk as soon as it is ready, but this can cause inter-chunk discontinuity. Temporal ensembling [17] smooths consecutive chunks by averaging overlapping actions, at the cost of blurring distinct modes. Bidirectional decoding (BID) [36] uses rejection sampling to maintain cross-chunk continuity, but at substantially higher computational cost.

Real-time chunking (RTC) [18] elegantly poses chunk transitions as inpainting via Π GDM [22] guidance, achieving strong performance on dynamic tasks. Following RTC, several works have focused on improving inference-time inpainting through better guidance design: ABPolicy [27] formulates the inpainting problem in a B-spline control-point action space for smooth manipulation. Orthogonally, another line of follow-up works focuses on training-time methods that directly learn to inpaint during fine-tuning: Training-Time RTC [23] conditions on action prefixes during training to internalize the inpainting capability; REMAC [24] follows a similar strategy but further eases the training–inference mismatch by mixing ground-truth and predicted actions during fine-tuning; Legato [25] learns a continuation-aware flow that reshapes the denoising dynamics to account for known prefixes at training time. FASTER [26] adopts a similar idea to rethink the indivisible characteristic of flow-matching training and inference, introducing a horizon-aware schedule to accelerate the time for the first available action. There are also works that bridge both stages: VLASH [37] achieves lower policy-input temporal mismatch by estimating future robot states and pre-training the VLA to predict action chunks conditioned on both current observations and future states.

Our method, DiscreteRTC, is the first effort to step outside the flow-matching policy framework and analyze the asynchronous execution problem for discrete diffusion policies. It requires no training-time modification, yet achieves strong performance by exploiting the native inpainting capability of discrete diffusion—and can be combined with methods like VLASH [37] for further gains.

B Additional Experiments

B.1 Natural Schedule Ablations

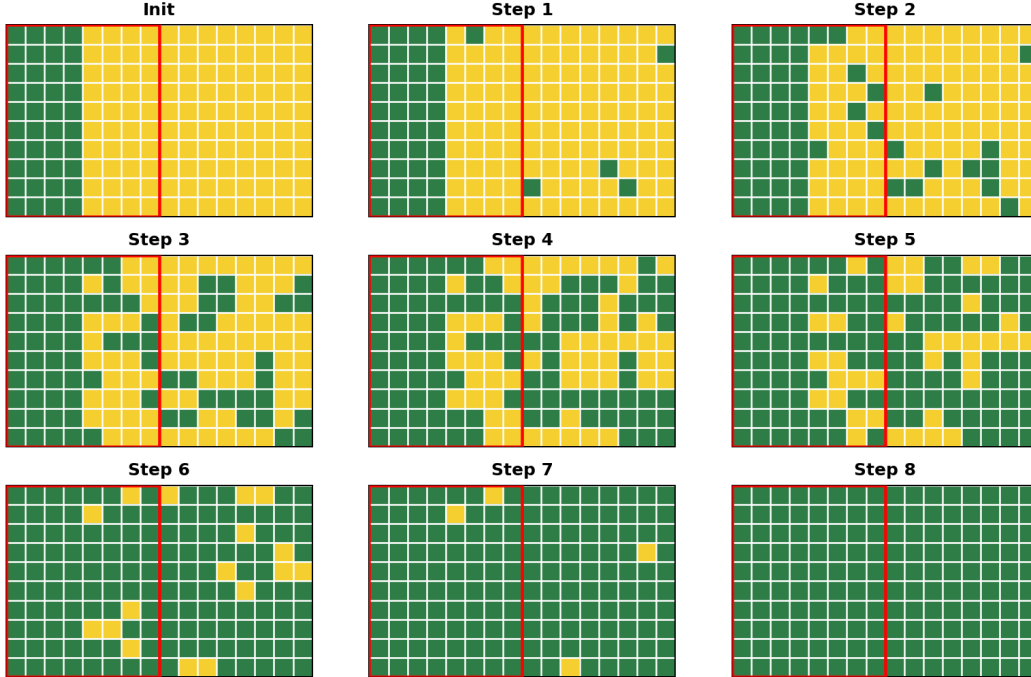


Figure 7: **Unmasking Trajectory Sample with Natural Schedule Inference.** Green blocks denote unmasked action tokens, yellow blocks denote masked tokens, and the red rectangle marks the early-stop boundary beyond which tokens do not need to be unmasked before the next inference.

In practice, the natural schedule does work as expected compared to the simple hard mask approach. In this section, we show how the inappropriate unmasking-schedule limits the performance of natural schedule. We visualize the per-step unmasking trajectory of the discrete diffusion policy under the max-confidence decoding strategy [28]. In the visualization, green blocks represent unmasked action tokens, yellow blocks represent masked tokens, and the red rectangle marks the early-stop boundary: only tokens inside this boundary need to be fully unmasked before the next inference, while tokens outside can remain partially masked and carry forward as the natural guidance signal. For each generation, the policy starts with the first $d = 4$ actions already unmasked (the frozen prefix from the previous chunk) and is expected to inpaint the remaining tokens in an implicitly autoregressive fashion—unmasking the early actions first so that execution can begin as soon as possible.

However, as shown in Figure 7, the max-confidence unmasking trajectory does *not* exhibit this desired implicit AR-style ordering. Instead, the policy distributes its unmasking budget across the entire chunk based on per-token confidence, and in most cases still consumes the full 8 unmasking steps before all tokens inside the early-stop boundary are resolved. This both neutralizes the inference-cost savings promised by early stopping and degrades follow-up generations: because more tokens are fully unmasked by the time inference completes, the next chunk inherits a larger committed prefix and correspondingly less flexibility from the natural guidance signal. This observation motivates the third direction in our future steps—designing a principled unmasking strategy (e.g., AR-block decoding) that aligns with the implicit autoregressive structure induced by RTC and fully unleashes the potential of the natural schedule.

B.2 Detailed Main Results in Kinetix

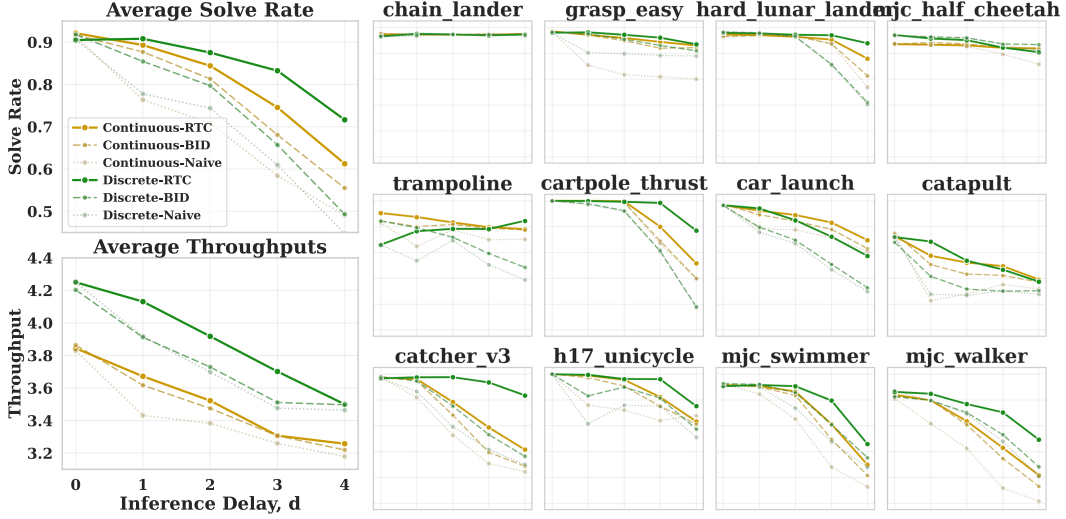


Figure 8: **Detailed Main Results in Kinetix.** The evaluation setup keeps the same with Figure 4. The small plots represent the results per Kinetix environment

B.3 Inpainting Fine-tuning

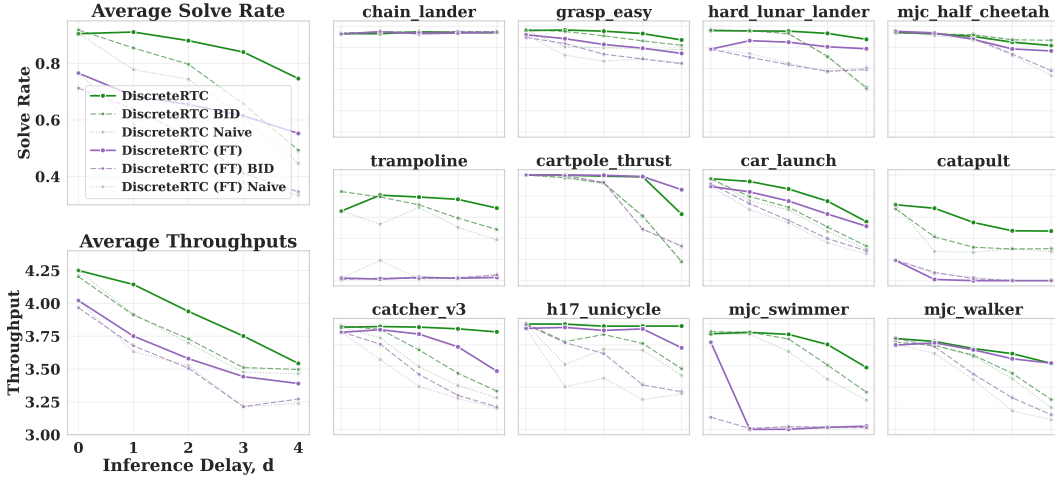


Figure 9: **Fine-tuning Ablation in Kinetix.** The evaluation setup keeps the same with Figure 4. The small plots represent the results per Kinetix environment

As an ablation of the fine-tuning-free claim, we explicitly fine-tune the pre-trained discrete diffusion policy with an inpainting-specific training recipe: for each training chunk we construct the target masking pattern by randomly masking as standard process first, then unmasking the prefix action tokens from $\{0, 1, 2, 3, 4\}$ to match the inference delay value distribution. The intention is to bias the base policy toward the RTC-style prefix–continuation and further improve inpainting quality.

However, as shown in Figure 8 and 9, this naive fine-tuning strategy brings a consistent performance drop across all 12 Kinetix levels and every inference delay. This suggests that a naive prefix-unmasking recipe can actually *hurt* the native inpainting capability of the pre-trained discrete diffusion policy. Improving beyond the fine-tuning-free DiscreteRTC therefore appears to require either a carefully designed inpainting-aware fine-tuning recipe that does not distort the pre-training distribution, or just directly scaling the base policy’s pre-training.

C Kinetix Implementation Details

The codebase is developed upon the [official RTC codebase](#) [18], implemented in JAX [51] with Flax NNx on 8 RTX 6000 PRO.

Environment. We use Kinetix [29] (Kinetix-Symbolic-Continuous-v1, large configuration). The physics simulation runs at 60 Hz with frame skip 2, yielding 30 Hz control. Actions are normalized to $[-1, 1]$ with Gaussian noise (std 0.1) added during both data collection and evaluation. Observations include a 4-frame history stack and the previous action.

Policy Architecture. Both policies share an MLP Mixer backbone with AdaLN conditioning: 4 blocks, each with token-mixing (hidden dim 64) and channel-mixing (hidden dim 512) paths, operating on $H=8$ tokens (one per chunk timestep) with channel dimension 256.

For **flow-matching policy**, the input is the observation concatenated with the noised action chunk. A sinusoidal time embedding conditions the AdaLN layers, and the output is a velocity vector. For **discrete diffusion policy**, actions are discretized into 512 bins via uniform quantization. The input is the observation concatenated with bin embeddings of the partially masked token sequence. The token sequence is packed from $(H, \text{action_dim}, \text{embedding_dim})$ to $(H, \text{channel_dim})$ via a linear layer so the mixer sees the same 8-token structure. A learned vector replaces the time conditioning.

Training. Both policies are trained for 32 epochs with batch size 512 using AdamW. The flow-matching policy uses a constant LR schedule and MSE loss on the predicted velocity field. The discrete diffusion policy uses cosine LR decay and cross-entropy on masked positions plus an auxiliary L1 loss ($\lambda=0.1$) with a cosine masking schedule. The training-time RTC is trained with the same setting with its simulated delay randomly sampled from the possible set, following the RTC codebase. The starting checkpoint is selected from epoch 24, and finetuned for 8 more epochs.

Evaluation. Both policies use 5 denoising/unmasking steps. For ContinuousRTC, IIGDM guidance with exponential-decay soft masking and clipping threshold β steers generation toward the frozen prefix. For DiscreteRTC, the frozen prefix tokens are directly placed, and for parallel evaluation across the environments, we use a hard mask for the chunk continuation. All remaining hyperparameters are listed in Table 1.

Table 1: Simulation experiment hyperparameters.

Hyperparameter	Flow-Matching	Discrete Diffusion
<i>Architecture</i>		
Channel / channel hidden / token hidden dim	256 / 512 / 64	
Number of MLP Mixer layers	4	
Action chunk size (H)	8	
Number of bins / embedding dim	—	512 / 128
<i>Training</i>		
Batch size / epochs	512 / 32	
Optimizer / LR / weight decay	AdamW / 3×10^{-4} / 10^{-2}	
Gradient norm clip	10.0	
LR schedule (warmup steps)	Constant (1,000)	Cosine decay (2,000)
Training loss	MSE (velocity)	CE + L1 ($\lambda=0.1$)
Training mask schedule	—	Cosine
<i>Evaluation</i>		
Denoising / unmasking steps	5	5
Decode temperature / choice temperature	—	1.0 / 0.0
RTC guidance	IIGDM ($\beta=5.0$)	Hard Mask
Inference delays (d)	{0, 1, 2, 3, 4}	
Execution horizon (s) / trials	$\max(1, d) / 2,048$	

D Real-world Implementation Details

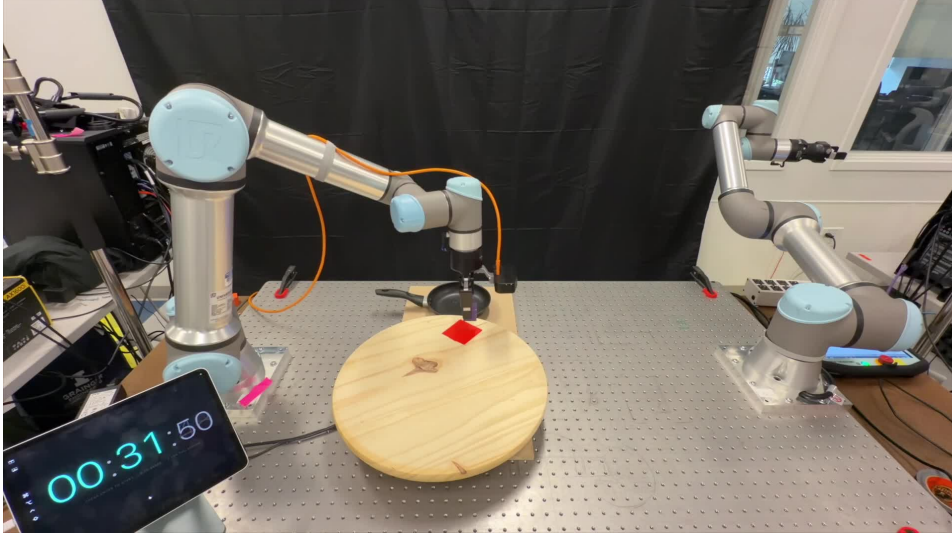


Figure 10: Dynamic Pick and Place Real-world Setup



Figure 11: Hockey Defend Real-world Setup

Hardware and Data. We use a single UR5e arm with a Robotiq gripper and a wrist-mounted RGB camera. Demonstrations are recorded at 500 Hz via the FastUMI pipeline. Each action is a 10D vector $[\Delta x, \Delta y, \Delta z, \text{rot6d}(6), \text{gripper}]$, with translational dimensions normalized to $[-1, 1]$ via min-max scaling, rotation dimensions left unnormalized, and the gripper binarized to $\{0, 1\}$. Images are center-cropped and resized to 224×224 . Training uses 8 RTX Pro 6000 with DeepSpeed ZeRO-2 and gradient checkpointing; deployment runs on a single RTX 4090.

Policy Architecture. All policies use Qwen2.5-VL-3B-Instruct [2] as the VLM backbone. Both action heads share the same QwenPI-style layerwise cross-attention DiT [39]: 36 blocks, each with AdaLN, cross-attention to the corresponding VLM hidden states, self-attention, and a feedforward network. Both prepend 32 learnable future vision tokens and add positional embeddings. When proprioceptive state is available, a two-layer MLP encodes it and prepends it to the sequence.

For **flow-matching head**, a three-layer MLP fuses per-timestep action embeddings with sinusoidal flow-time embeddings (output dim 2048); a sinusoidal `TimestepEncoder` provides AdaLN conditioning; and a two-layer MLP decodes the velocity output. For **discrete diffusion head**, actions are quantized into 256 bins per dimension, yielding $16 \times 10 = 160$ tokens per chunk. Tokens (including a learnable [MASK]) are embedded via `nn.Embedding(257, 2048)` with no timestep conditioning. For the training-time RTC, we select the final checkpoint to be continuously trained for another 2500 steps with the same setting. The simulated delay is pre-fixed with the estimated delays, $d = 4$ for the dynamic pick & place and $d = 6$ for the hockey defend. The RTC parameters here and below are selected to match the exact inference delay in deployment instead of manually adjusting.

Dynamic Pick & Place Deployment. The robot operates in closed-loop at 20 Hz with $5 \times$ linear interpolation producing a 100 Hz servo stream. Camera capture and model inference run in a background thread, overlapping with action execution. The rotation speed of the turntable for dynamic pick and place are 15 and 20 rpm. The discrete diffusion head uses 8 MaskGIT unmasking steps with a cosine decode schedule (decode temperature 0.0, choice temperature 0.1). By default we use non-fixed unmasking steps for the discrete diffusion policies and running RTC with $d = 4$, $s = 4$ for discrete diffusion policies, $d = 5$, $s = 5$ for flow-matching policies.

Air Hockey Defend Deployment. The robot operates in closed-loop at 50 Hz with $2 \times$ linear interpolation producing a 100 Hz servo stream to match the higher inference speed requirements. The hardware setup shares the same with the Dynamic Pick & Place. By default we use non-fixed unmasking steps for the discrete diffusion policies and running RTC with $d = 7$, $s = 7$ for discrete diffusion policies, $d = 9$, $s = 9$ for flow-matching policies.

All hyperparameters are summarized in Table 2.

Table 2: Real-world experiment hyperparameters.

Hyperparameter	Flow-Matching	Discrete Diffusion
<i>VLM Backbone & Action Head</i>		
VLM backbone	Qwen2.5-VL-3B-Instruct	
Action head	Layerwise Cross-Attention DiT	
DiT layers / heads / head dim	36 / 32 / 64 (inner dim 2048)	
Dropout / future vision tokens	0.2 / 32	
Action encoder	3-layer MLP + sinusoidal time	Embedding(257, 2048)
Action decoder	2-layer MLP (hidden 1024)	Linear $\rightarrow$ 256 logits
Timestep conditioning	TimestepEncoder (AdaLN)	None (zero vector)
<i>Action Representation</i>		
Action dim	10 (Pick & Place) / 2 (Hockey)	
chunk length H	16 (Pick & Place) / 32 (Hockey)	
Representation	Continuous	256-bin quantization
<i>Training</i>		
Optimizer	AdamW ($\beta_1=0.9, \beta_2=0.95$)	
LR (VLM / action model)	$10^{-5} / 10^{-4}$	
LR scheduler (warmup / total steps)	Cosine, min 5×10^{-7} (5k / 30k)	
Batch size (per device $\times$ GPUs)	$8 \times 8 = 64$	
Mixed precision	bfloat16 (VLM) / float32 (action head)	
Training loss	MSE (velocity)	CE + L1 ($\lambda=0.1$)
Noise / mask schedule	Beta(1.5, 1.0), $s=0.999$	Cosine
<i>Inference & Deployment</i>		
Denoising / unmasking steps	8	8
Decode / choice temperature	—	0.0 / 0.1
Control / servo frequency	20 Hz / 100 Hz ($5 \times$ interpolation)	
Robot / camera	UR5e + Robotiq / wrist-mounted fish-eye RGB	